



SISTEMAS OPERATIVOS - TEORÍA
14 de junio de 2022

Nombre _____ DNI _____
Apellidos _____ Grupo _____

Cuestión 1. (2 puntos) Considere un sistema de ficheros basado en nodos-i en el que se utiliza un tamaño de bloque de 2KiB, un tamaño de puntero a bloque de 32 bits, y donde un nodo-i está formado por cuatro índices directos, 1 índice indirecto simple y 1 índice indirecto doble.

Un usuario crea un directorio en ese sistema, y sobre ese nuevo directorio (usándolo como directorio de trabajo) ejecuta secuencialmente los siguientes programas (primero programA y luego programB), que ya se encuentran compilados en otro directorio del sistema:E

```
// programA
#include ...
#define BUFFER_SIZE 512

char buf[BUFFER_SIZE]="yyyyyy...y";

int main( void )
{
    int fd;

    fd=open( "file.txt", O_WRONLY | O_CREAT
              | O_TRUNC, 0644 );
    write(fd,"x",1);
    lseek(fd, 8*BUFFER_SIZE, SEEK_CUR);
    write(fd,buf,BUFFER_SIZE);
    return 0;
}
```

```
// programB
#include ...
#define BUFFER_SIZE 512

int main( void )
{
    int fd;
    char c;

    symlink("file.txt","anotherfile");
    fd=open("anotherfile", O_RDWR);
    lseek(fd, 16*BUFFER_SIZE, SEEK_END);
    write(fd,"z",1);
    lseek(fd, 0, SEEK_SET );
    read(fd, &c, 1);
    printf("%c\n",c);
    return 0;
}
```

Responda **razonadamente** a las siguientes preguntas, asumiendo que todas las llamadas al sistema que invocan los programas tienen éxito (no se valorarán respuestas no razonadas):

- a. ¿Cuántos ficheros distintos se crean en el directorio? ¿Qué se imprime por pantalla tras ejecutar ambos programas?

- b. ¿Cuántos bloques ocupa en disco cada uno de los ficheros creados?

- c. Suponga que *programA* únicamente se ejecutará satisfactoriamente en un sistema de ficheros con asignación enlazada (p.ej., tipo FAT), empleando el mismo tamaño de bloque, y trabajando sobre un directorio vacío. ¿Cuántos bloques ocuparía en disco el fichero creado por ese programa?

Cuestión 2. (2 puntos) Considere un sistema monoprocesador con una política de planificación de procesos de 2 niveles con realimentación. Ambos niveles usan una política de planificación expropiativa *circular* (round robin) con cuantos de tiempo 2 y 3, respectivamente. Al principio hay 3 procesos en la cola del nivel L1: P1, P2 y P3, en este orden. Los patrones de ejecución de los procesos son los siguientes (y se repiten indefinidamente):

P1 (5-CPU,3-E/S), P2 (4-CPU,3-E/S), P3 (5-CPU, 4-E/S)

La cola L2 está inicialmente vacía. Cuando un proceso agota su tiempo de CPU, pasa a la cola de nivel inferior. Cuando acaba una operación de E/S, los procesos entran en la cola de mayor prioridad. Usando el diagrama de tiempos de la figura, muestre:

- Qué proceso está ejecutándose (utilice 'X1' o 'X2' para marcar el proceso en ejecución desde el nivel L1 ó L2, respectivamente, y 'O' para marcar el proceso bloqueado por E/S) y qué procesos hay en cada nivel (utilice las filas de L1 para el primer nivel y las de L2 para el segundo), durante las 20 primeras unidades de tiempo.
- Calcule el porcentaje de utilización de CPU y los tiempos de espera de cada proceso.

	00	01	02	03	04	05	06	07	08	09	10	11	12	13	14	15	16	17	18	19	20
P1(5,3)	X1																				
P2(4,3)																					
P3(5,4)																					
L1	P2																				
RR(2)	P3																				
L2																					
RR(3)																					

Porcentaje de uso de CPU: _____ %

Tiempo de espera de P1: _____ unidades

Tiempo de espera de P2: _____ unidades

Tiempo de espera de P3: _____ unidades

Cuestión 3. (2 puntos) Se desea diseñar el sistema de control del aforo de una discoteca. En la discoteca solo pueden estar un máximo de N ocupantes, que entran y salen de uno en uno. Hay dos tipos de clientes de la discoteca: los clientes-vip y los normales. El comportamiento del sistema es el siguiente:

Si el número de ocupantes de la discoteca es menor que N, los que estén esperando entrar pasarán de uno en uno en orden de llegada.

En caso contrario, deberán esperar a que salgan clientes. Los clientes-vip entrarán antes que los normales, en caso de que haya que esperar para entrar.

Cliente-vip() {	Cliente-normal() {
Entrar-vip();	Entrar-normal();
Bailar();	Bailar();
Salir();	Salir();
}	}

Se pide implementar las funciones Entrar-vip, Entrar-normal y Salir usando mutex y variables condicionales.

Cuestión 4. (2 puntos) En un sistema tipo UNIX, con memoria virtual con paginación bajo demanda y páginas de 4KiB, se compila y enlaza estáticamente el siguiente código.

```
#include <stdlib.h>
#include <stdio.h>
#include <unistd.h>
#include <pthread.h>
#include <string.h>
#include <sys/mman.h>
#include <sys/wait.h>

#define N 10
int num = 0;
int *buffer;

void *suma(void *arg)
{
    int sum = 0, i;
    for (i = 0; i < N; i++)
        sum += buffer[i];
    // Punto E
    printf("sum is: %d\n", sum);
    return NULL;
}

int main(int argn, char *argv[])
{
    pid_t pid;
    int i;
    pthread_t tid;
    printf("num is: %d\n", num);
    // Punto A
    buffer = mmap(NULL, N*sizeof(int), PROT_READ|PROT_WRITE, MAP_SHARED|MAP_ANONYMOUS, 0, 0);
    // Punto B
    if ((pid = fork()) == -1) {
        perror("fork");
        exit(EXIT_FAILURE);
    }
    if (pid == 0) {
        num++;
        printf("num is: %d\n", num);
        // Punto C
        for (i = 0; i < N; i++)
            buffer[i] = i+1;
        // Punto D
        execlp("echo", "echo", "Hello world", NULL);
        perror("exec");
        exit(-1);
    }

    while (wait(NULL) != -1);
    printf("num is: %d\n", num);
    pthread_create(&tid, NULL, suma, NULL);
    pthread_join(tid, NULL);
    munmap(buffer, N*sizeof(int));
    return 0;
}
```

Responda a las siguientes preguntas:

- a) Completa la siguiente tabla, indicando para cada símbolo la región de memoria en la que aparece en el mapa de memoria de ambos procesos, la protección de dicha región y si tiene soporte en el fichero ejecutable o no lo tiene:

Símbolo	Región de Memoria Padre	Región de Memoria Hijo	Protección (RWX)	Soporte en Ejecutable (si/no)
suma				
pid				
buffer				
sum				

- b) Indica las regiones de memoria que existen en los espacios de direcciones de los procesos cuando el hijo está en la línea marcada con el comentario PUNTO C.

- c) Describe el efecto que tiene en el mapa de memoria del proceso hijo la llamada a `execvp` tras el punto D.

- d) Describe los cambios que se producen, respecto del apartado anterior, en el mapa de memoria del proceso padre hasta que éste llega al punto E.

- e) Indica razonadamente cuántos fallos de página se producen cuando el proceso pasa del PUNTO A al PUNTO B y del PUNTO C al PUNTO D

Cuestión 5 (2 puntos) En un sistema Linux se ejecuta el siguiente código, suponiendo que en la sección #include están todas las librerías necesarias y todas las llamadas al sistema tienen éxito.

<pre> 1 #include ... 2 > void* thread1(void* arg) { ... 5 } 6 > void* thread2(void* arg) { ... 9 } 10 11 int main(){ 12 13 int status; 14 pid_t pid, pt_id[7]; 15 16 pt_id[0] = getpid(); 17 pid = fork(); 18 if (pid == 0){ 19 pthread_t tid1, tid2; 20 pthread_create(&tid1,NULL,thread1,NULL); 21 pthread_create(&tid2,NULL,thread2,NULL); 22 pthread_join(tid1,NULL); 23 pthread_join(tid2,NULL); 24 exit(EXIT_SUCCESS); 25 } 26 pt_id[1] = pid; 27 waitpid(pt_id[1], &status, 0); 28 29 pid = fork(); 30 if (pid == 0){ 31 pthread_t tid; 32 pthread_create(&tid,NULL,thread1,NULL); 33 pthread_join(tid,NULL); 34 35 exit(EXIT_SUCCESS); 36 } 37 pt_id[2] = pid; 38 pid = fork(); 39 if (pid == 0){ 40 pthread_t tid; 41 pthread_create(&tid,NULL,thread1,NULL); 42 pthread_join(tid,NULL); 43 44 exit(EXIT_SUCCESS); 45 } 46 pt_id[3] = pid; 47 </pre>	<pre> 47 48 pid = fork(); 49 if (pid == 0){ 50 # Working 51 exit(EXIT_SUCCESS); 52 } 53 pt_id[4] = pid; 54 55 waitpid(pt_id[2], &status, 0); 56 waitpid(pt_id[3], &status, 0); 57 58 pid = fork(); 59 if (pid == 0){ 60 pthread_t tid; 61 pthread_create(&tid,NULL,thread1,NULL); 62 pthread_join(tid,NULL); 63 exit(EXIT_SUCCESS); 64 } 65 pt_id[5] = pid; 66 waitpid(pt_id[4], &status, 0); 67 waitpid(pt_id[5], &status, 0); 68 69 pid = fork(); 70 if (pid == 0){ 71 pthread_t tid1, tid2; 72 pthread_create(&tid1,NULL,thread1,NULL); 73 pthread_create(&tid2,NULL,thread2,NULL); 74 pthread_join(tid1,NULL); 75 pthread_join(tid2,NULL); 76 exit(EXIT_SUCCESS); 77 } 78 pt_id[6] = pid; 79 80 waitpid(pt_id[6], &status, 0); 81 return 0; 82 } 83 </pre>
--	---

- Describir y dibujar en forma de árbol la secuencia de procesos con sus threads correspondientes.
- ¿Cuál es el número máximo de procesos ejecutándose en paralelo? ¿Y threads?
- ¿Qué información se guarda en el array pt_id?

